

Muhammad Farooqi — Interview Prep Cheat Sheet

WATonomous — Perception Software Engineer

Q1: What is a voxel and why use a voxel grid instead of a raw point cloud?

Interviewer question:

What is a voxel, and why would you use a voxel grid instead of just keeping the raw point cloud for downstream robotic perception tasks?

Model Answer:

A voxel is a 3D unit of volume — like a 3D pixel. I discretize the point cloud by binning 3D points into a regular grid where each cell is a fixed size (in my case 4cm x 4cm x 4cm). If points fall in the same cell, they get grouped into that voxel.

The reason you use a voxel grid over a raw point cloud is structure and efficiency. A raw point cloud is unordered — you can't directly apply convolutions to it. A voxel grid gives you a regular spatial structure that neural networks can process. And if you use sparse storage, you only keep the voxels that actually have points in them, which in a typical scene is a tiny fraction of the total possible grid cells.

Q2: What does sparse mean in sparse voxel grid?

Interviewer question:

What is a sparse voxel grid vs a dense voxel grid and why does sparsity matter for performance?

Model Answer:

A dense grid allocates memory for every possible voxel cell in the 3D space — even the empty ones. If your scene covers a 100m range with 4cm voxels, you're talking about tens of millions of cells, the vast majority of which are just empty air.

Sparse storage means you only store the voxels that actually have points in them. In a typical scene that's a tiny fraction — maybe 2,000 occupied voxels out of 30 million possible ones. Everything else doesn't exist in memory at all.

Q3: What does spconv's PointToVoxel actually return?

Interviewer question:

In your create_voxel_grid function, voxel_gen() returns three things. What are they and what does each contain?

Model Answer:

voxels — shape [M, max_points_per_voxel, num_features]. For each of the M occupied voxels, it stores up to 20 raw points (my max_num_points_per_voxel=20), each with 3 features (x, y, z). So it's not just an

occupied flag — it preserves the actual point geometry inside each voxel for downstream feature learning.

indices — shape [M, 3]. The integer grid coordinates (z, y, x) of each occupied voxel. I use these to recover real-world meter positions: $\text{centers} = \text{origin} + (\text{idx_xyz} + 0.5) * \text{voxel_size}$.

num_points — shape [M]. How many points actually landed in each voxel, since most won't have the full 20.

Q4: Why does Kodiak specifically care about sparse voxel grids?

Interviewer question:

Why would Kodiak specifically care about sparse voxel grids on an autonomous truck?

Model Answer:

A LiDAR on a highway truck shoots millions of points per second but most of that space is empty air. If you stored a dense grid covering the truck's full perception range — say 100m ahead — you'd have an astronomically large grid that's 99% empty. Sparse storage means you only process the small fraction of voxels that actually contain returns from real objects like cars, barriers, or road signs.

That makes the pipeline fast enough to run in real time at highway speeds — at 26 m/s you have maybe 50-100ms per frame to make a perception decision. Dense grids simply can't meet that latency budget.

■ *Key numbers to remember: 60mph = 26.82 m/s. Real-time perception budget = 50-100ms per frame.*

Q5: Are you actually doing RGB-D fusion? (The Trap Question)

Interviewer question:

Your resume says RGB-D fusion — walk me through what that means and whether you're actually fusing RGB and depth in your pipeline.

Model Answer:

RGB-D fusion means combining the color information (RGB — a 640x480x3 image) with the depth channel (640x480 depth values) so that each pixel has both color and depth. Because the width and height are the same, every pixel in the RGB image has a corresponding depth value in the depth image.

The fusion happens during the unprojection step — when I convert pixel (u, v) with depth z into a 3D point using the pinhole camera model: $x = (u - c_x) * z / f_x$, $y = (v - c_y) * z / f_y$. That's where the two modalities come together spatially.

In the current implementation, points are output as (x, y, z) only. The full RGB-D fusion — attaching the RGB color at pixel (u, v) to each 3D point to get (x, y, z, r, g, b) — is the next step in the pipeline. The infrastructure is in place: both topics are subscribed to and both images are decoded. The color attachment is the natural extension.

■ *Don't oversell this. If pressed, be honest that current output is (x,y,z) only. Showing you know exactly what full fusion looks like is more impressive than bluffing.*

Q6: The 4x speedup claim — how did you measure it?

Interviewer question:

You claim 4x runtime speedup via stride-based downsampling. How did you actually measure that?

Model Answer:

With a stride of 4 in both x and y directions, I sample 1 out of every 16 pixels. The bottleneck is the per-point operations downstream — the pinhole unprojection and voxelization — so reducing the point count by 16x gives approximately a 4x effective speedup on the full pipeline. I benchmarked this using `time.time()` measurements before and after the pipeline, running `stride=1` vs `stride=4` over multiple frames and comparing average processing time.

■ *Fill in the actual measured numbers here once you run the benchmark. Replace '4x' with your real multiplier if it differs.*

Q7: Why does Dockerizing a ROS2 stack matter specifically?

Interviewer question:

Why does Dockerizing a ROS2 stack matter specifically — what actually breaks without it when multiple people are working on the same codebase?

Model Answer:

ROS2 is particularly painful without Docker because it has tight version coupling across the entire stack. `spconv` requires a specific version of PyTorch and CUDA to compile correctly — if someone has CUDA 11 and you built against CUDA 12, it won't run. ROS2 itself has version-specific message formats and build tools — mixing ROS2 Humble and Foxy on the same machine causes conflicts. And Isaac Sim has its own container with its own CUDA and Python requirements that clash with everything else.

Without Docker, getting all three stacks — perception, simulation, and behaviour — running on a new machine could take days of debugging. In our monorepo each stack has its own Dockerized environment and you just `docker compose up`. That's what reproducible actually means in practice.

■ *Key specifics: spconv + CUDA version coupling, ROS2 Humble vs Foxy conflicts, Isaac Sim container requirements, monorepo with separate containers per stack.*

Q8: What do you actually lose with stride-based downsampling?

Interviewer question:

Explain exactly what stride-based downsampling is doing geometrically — not just 'we skip pixels.' What are you actually sampling, what do you lose, and why is that tradeoff acceptable?

Model Answer:

Instead of processing every pixel in the depth image, I sample every 4th pixel in both x and y directions using `np.arange(0, h, stride)` and `np.meshgrid`. This reduces the number of points fed into the pipeline by roughly 16x — from potentially 300,000 points for a 640x480 image down to around 19,000.

What you lose geometrically is fine-grained detail on small or distant objects. With stride=4, two adjacent sample points are 4 pixels apart — at a depth of 3 meters with $f_x=525$, that's roughly 2-3cm of spatial resolution per sample. Any object smaller than that gap risks having zero points land in its voxels and becoming invisible to the pipeline.

In practice on a highway that's acceptable — the smallest relevant object like a cyclist or road debris is still large enough to produce multiple occupied voxels even at stride=4. Where it would break is in close-range high-precision tasks — like a lab robot doing dextrous manipulation at centimeter scale, which is exactly what Multiply Labs is doing.

■ *Key numbers: stride=4 reduces points ~16x. At 3m depth with $f_x=525$, stride=4 gives ~2-3cm spatial resolution gap. 640x480 full = ~300k points, stride=4 = ~19k points.*

Q9: Why does depth-range filtering matter?

Interviewer question:

Your code has $min_range=0.1$ and $max_range=3.5$. Why those values? What real-world problems does depth-range filtering solve and what happens if you remove it?

Model Answer:

Depth-range filtering removes points outside 0.1m to 3.5m. The minimum range of 0.1m removes points too close to the camera — at very short distances depth sensors become unreliable and produce noisy or invalid readings. The maximum range of 3.5m removes background points too far away to be useful for close-range perception.

Without this filter, the pipeline would try to voxelize noise hits near zero depth and distant background clutter — both waste compute and pollute the voxel grid with irrelevant data, slowing down the pipeline and reducing perception quality.

Q10: What are the limitations of your pipeline?

Interviewer question:

What are the limitations of your pipeline? What would actually break if you tried to deploy it on a real autonomous vehicle or a real lab robot at Multiply Labs?

Model Answer:

1. Single camera, no LiDAR — my pipeline is depth-only from a single RGB-D camera. Real AVs use LiDAR which has 360 degree coverage and much longer range, so my perception would be completely blind outside the camera's field of view.
2. No temporal fusion — each frame is processed independently with no memory of previous frames, so fast-moving objects would appear to jump around rather than being tracked smoothly.
3. Short max range — the 3.5m max range is fine for a lab robot but useless on a highway where you need to see 100m ahead.

4. Camera frame not world frame — I'm not doing any coordinate frame transforms, so as the robot moves the entire grid shifts, making it useless for building a persistent map.

5. Voxel size too coarse for Multiply Labs — my 4cm voxel size is too coarse for dextrous manipulation. Pipetting or handling small lab containers needs millimeter-level precision, not centimeter-level.

■ *Remember the 5 limitations: (1) single camera no LiDAR (2) no temporal fusion (3) short max range (4) camera frame not world frame (5) voxel size too coarse for Multiply Labs.*